CSE 2031
Advanced Algorithms

Greedy Algorithms (Part II)

Greedy Algorithm

- Greedy algorithms make the choice that looks best at the moment.
- This locally optimal choice may lead to a globally optimal solution (i.e. an optimal solution to the entire problem).

When can we use Greedy algorithms?

We can use a greedy algorithm when the following are true:

1) **The greedy choice property:** A globally optimal solution can be arrived at by making a locally optimal (greedy) choice.

2) **The optimal substructure property:** The optimal solution contains within its optimal solutions to subproblems.

Designing Greedy Algorithms

1. Cast the optimization problem as one for which:
   - we make a choice and are left with only one subproblem to solve
2. Prove the GREEDY CHOICE
   - that there is always an optimal solution to the original problem that makes the greedy choice
3. Prove the OPTIMAL SUBSTRUCTURE:
   - the greedy choice + an optimal solution to the resulting subproblem leads to an optimal solution

An Activity Selection Problem (Conference Scheduling Problem)

- **Input: A set of activities S = {a_1,…, a_n}**
- Each activity has start time and a finish time
  - a_i=(s_i, f_i)
- Two activities are compatible if and only if their interval does not overlap
- **Output: a maximum-size subset of mutually compatible activities**

The Activity Selection Problem

- Here are a set of start and finish times

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

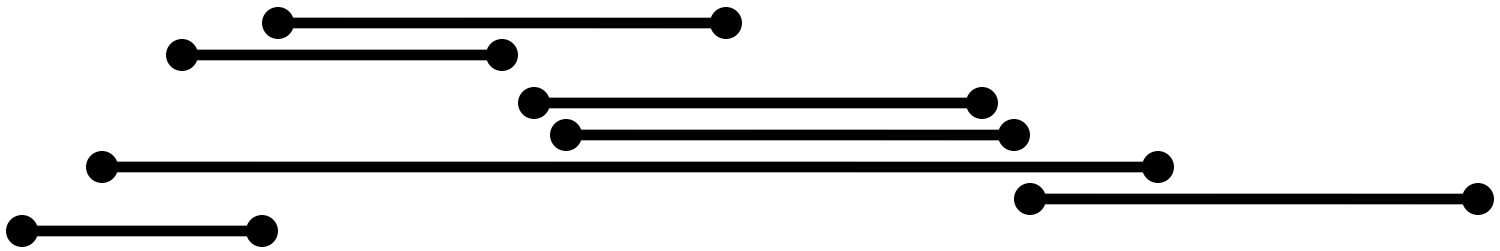
- What is the maximum number of activities that can be completed?
  - $\{a_3, a_9, a_{11}\}$ can be completed
  - But so can $\{a_1, a_4, a_8, a_{11}\}$ which is a larger set
  - But it is not unique, consider $\{a_2, a_4, a_9, a_{11}\}$

The Activity Selection Problem

Input: list of time-intervals L

Output: a non-overlapping subset S of the intervals

Objective: maximize |S|



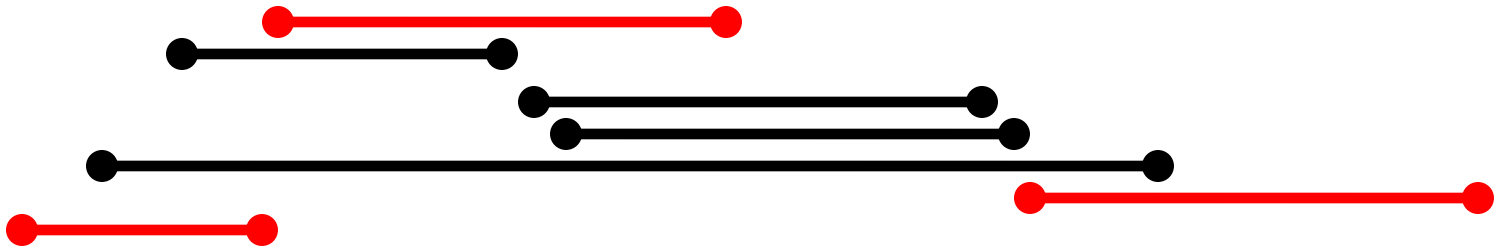
3,7
2,4
5,8
6,9
1,11
10,12
0,3

The Activity Selection Problem

Input: list of time-intervals L

Output: a non-overlapping subset S of the intervals

Objective: maximize |S|



Answer = 3

3,7
2,4
5,8
6,9
1,11
10,12
0,3

The Activity Selection Problem

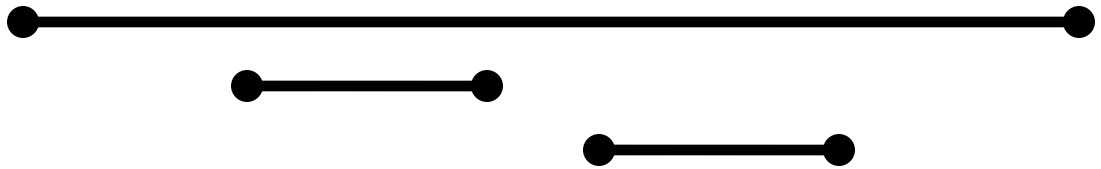
Algorithm 1:

1. sort the activities by the starting time
2. pick the first activity ***"a"***
3. remove all activities conflicting with ***"a"***
4. repeat

The Activity Selection Problem

Algorithm 1:

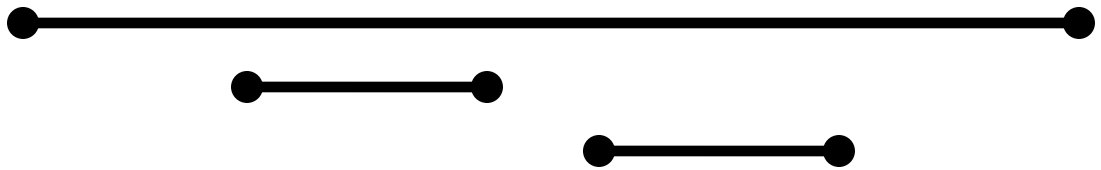
1. sort the activities by the starting time
2. pick the first activity ***"a"***
3. remove all activities conflicting with ***"a"***
4. repeat



The Activity Selection Problem

Algorithm 1:

1. sort the activities by the starting time
2. pick the first activity ***"a"***
3. remove all activities conflicting with ***"a"***
4. repeat



The Activity Selection Problem

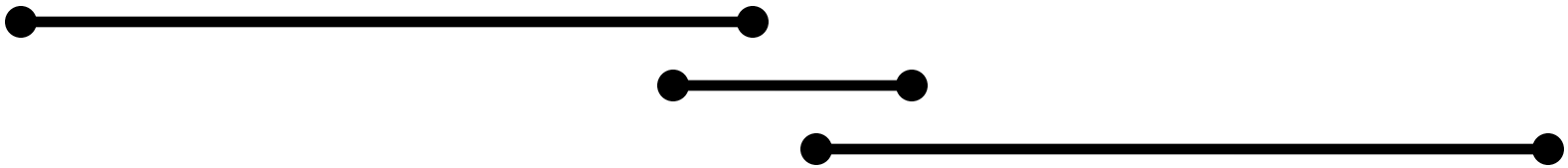
Algorithm 2:

1. sort the activities by length
2. pick the shortest activity ***"a"***
3. remove all activities conflicting with ***"a"***
4. repeat

The Activity Selection Problem

Algorithm 2:

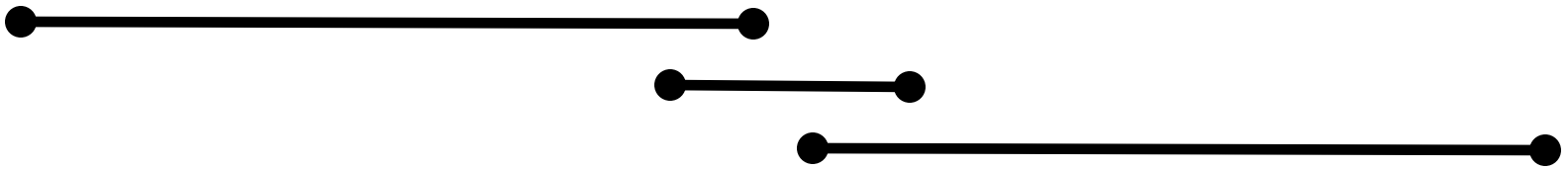
1. sort the activities by length
2. pick the shortest activity ***"a"***
3. remove all activities conflicting with ***"a"***
4. repeat



The Activity Selection Problem

Algorithm 2:

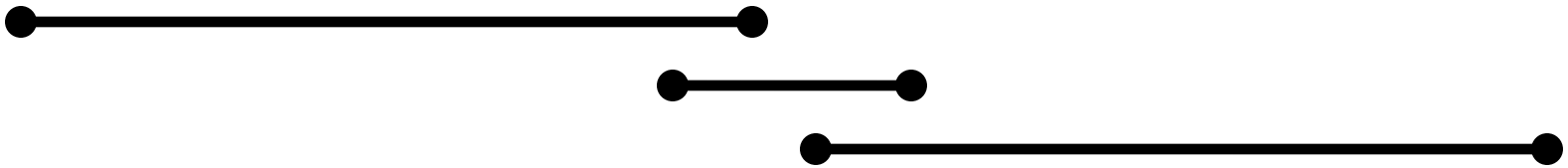
1. sort the activities by length
2. pick the shortest activity ***"a"***
3. remove all activities conflicting with ***"a"***
4. repeat



The Activity Selection Problem

Algorithm 3:

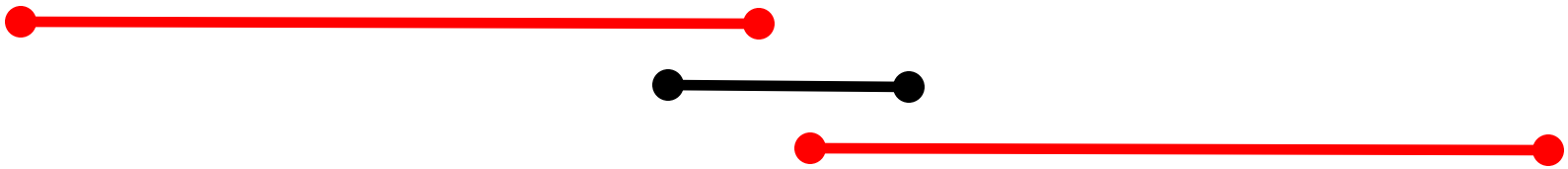
1. sort the activities by ending time
2. pick the activity which ends first
3. remove all activities conflicting with a
4. repeat



The Activity Selection Problem

Algorithm 3:

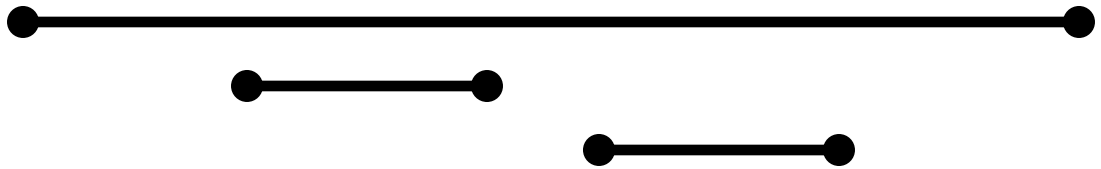
1. sort the activities by ending time
2. pick the activity which ends first
3. remove all activities conflicting with a
4. repeat



The Activity Selection Problem

Algorithm 3:

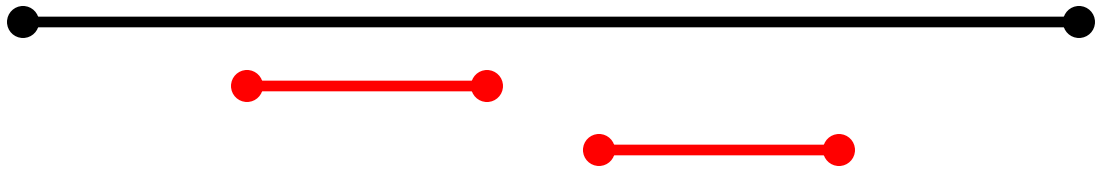
1. sort the activities by ending time
2. pick the activity which ends first
3. remove all activities conflicting with a
4. repeat



The Activity Selection Problem

Algorithm 3:

1. sort the activities by ending time
2. pick the activity which ends first
3. remove all activities conflicting with a
4. repeat



The Activity Selection Problem

Algorithm 3:

1. sort the activities by ending time
2. pick the activity “a” which ends first
3. remove all activities conflicting with “a”
4. repeat

Theorem:

Algorithm 3 gives an optimal solution to the activity selection problem.

Activity Selection Algorithm

Idea: At each step, select the activity with the smallest finish time that is compatible with the activities already chosen.

```
Greedy-Activity-Selector(s, f)
    n <– length[s]
    A <– {1}                    {Automatically select first activity}
    j <– 1                      {Last activity selected so far}
    for i <– 2 to n do
        if si >= fj then
            A <– A U {i}        {Add activity i to the set}
            j <– i              {record last activity added}
    return A
```

The Activity Selection Problem

- Here are a set of start and finish times

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

- What is the maximum number of activities that can be completed?
  - $\{a_3, a_9, a_{11}\}$ can be completed
  - But so can $\{a_1, a_4, a_8, a_{11}\}$ which is a larger set
  - But it is not unique, consider $\{a_2, a_4, a_9, a_{11}\}$

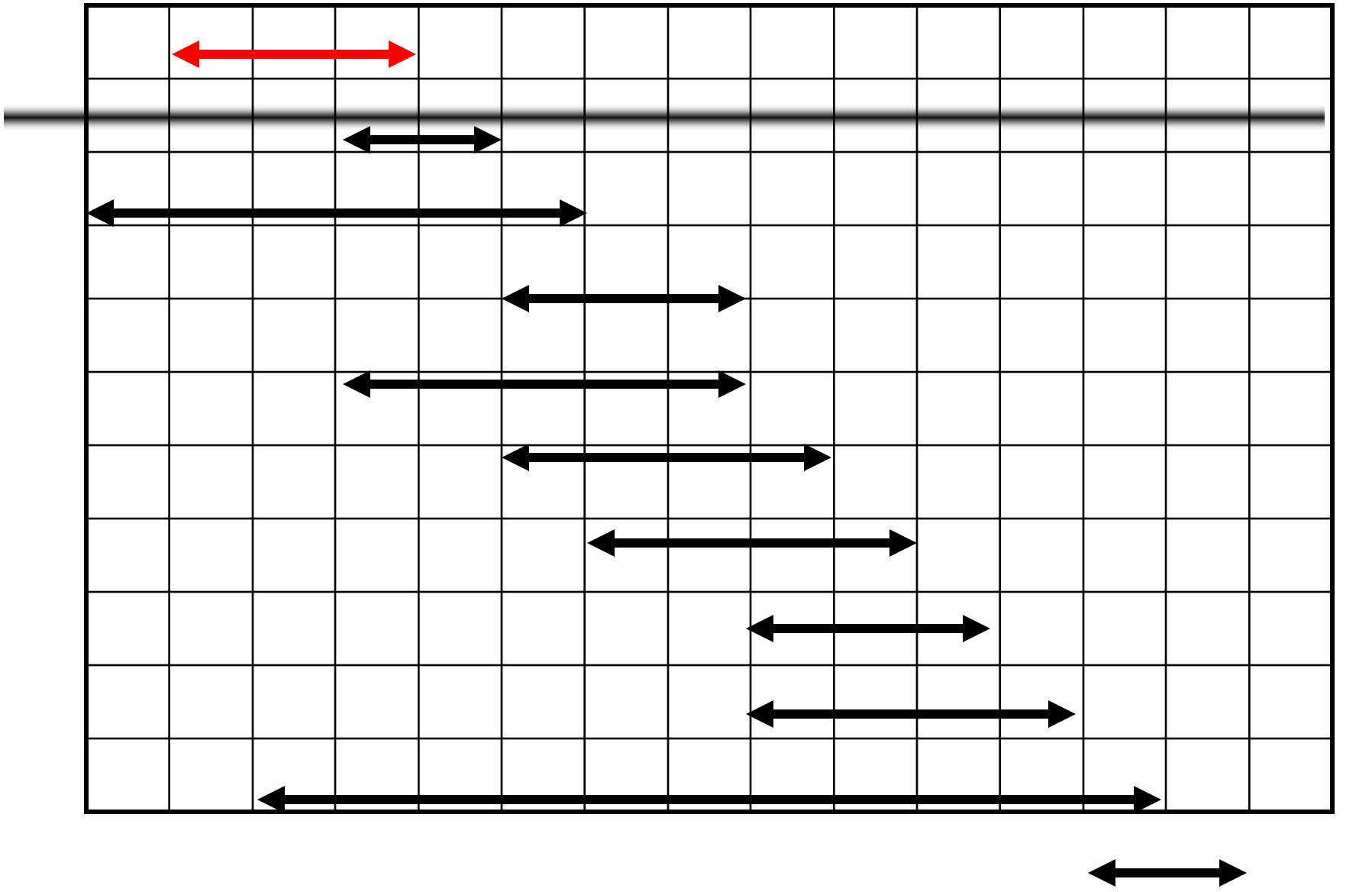
Interval Representation

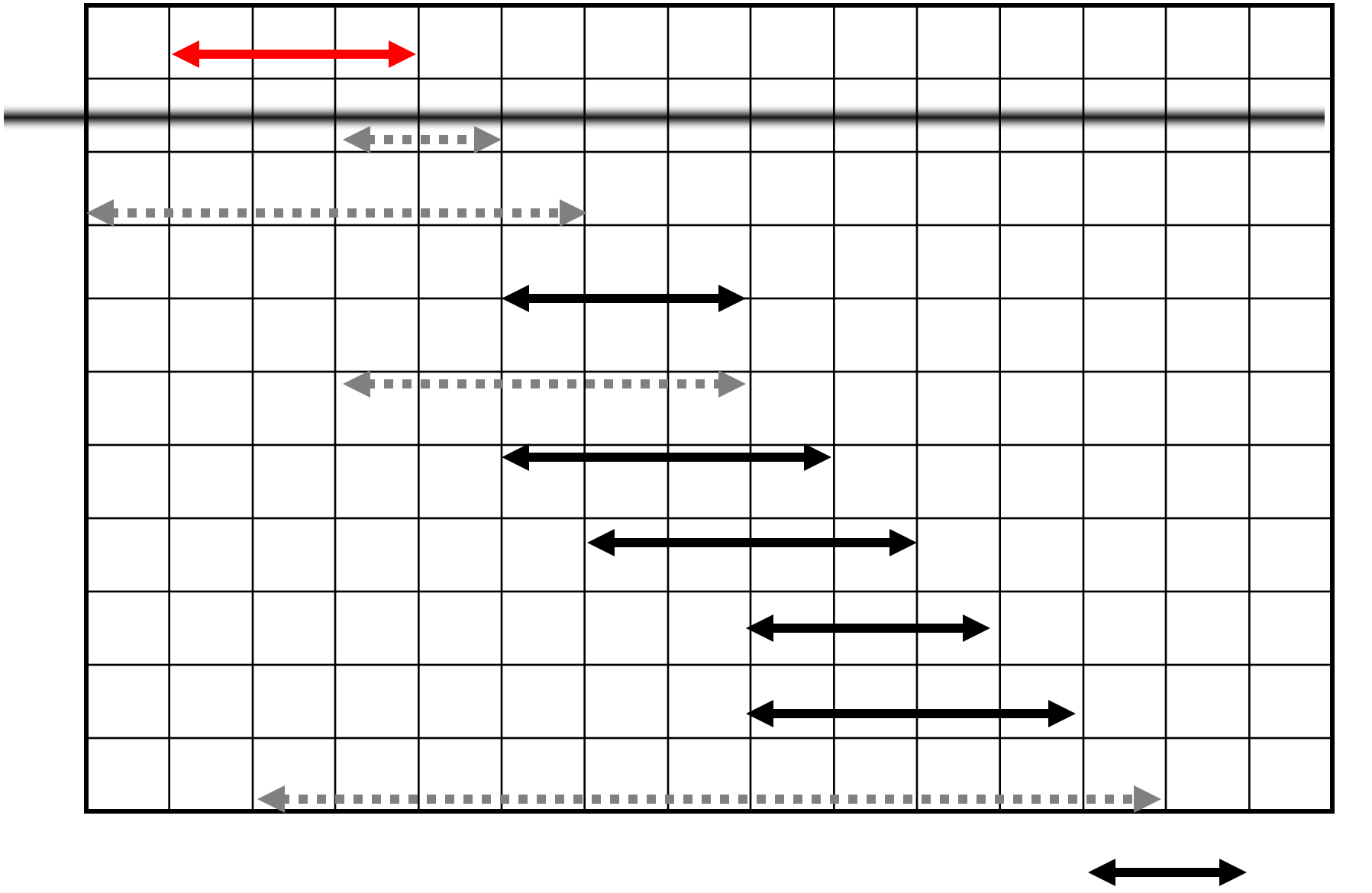
i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

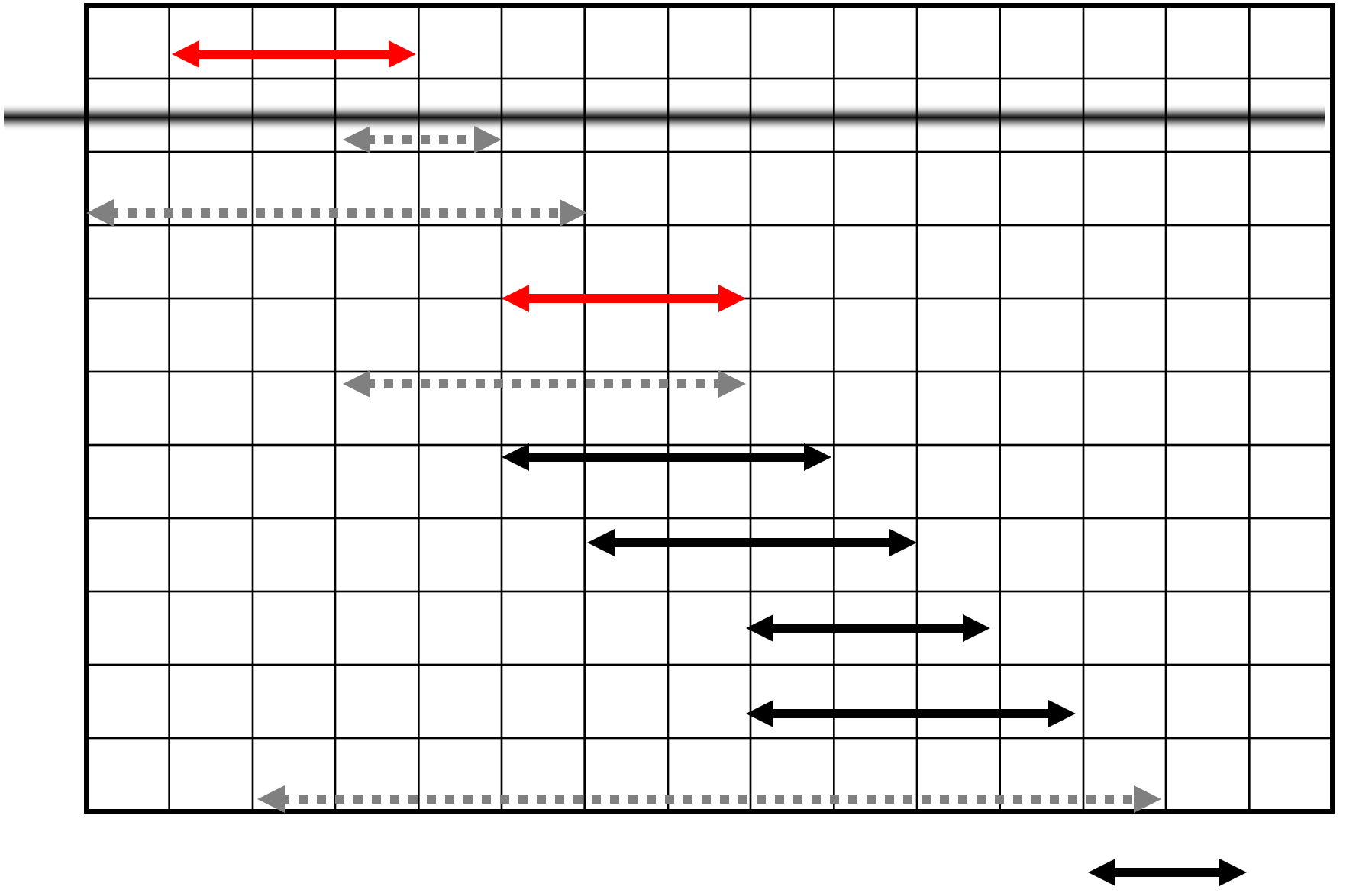
Not Observed yet

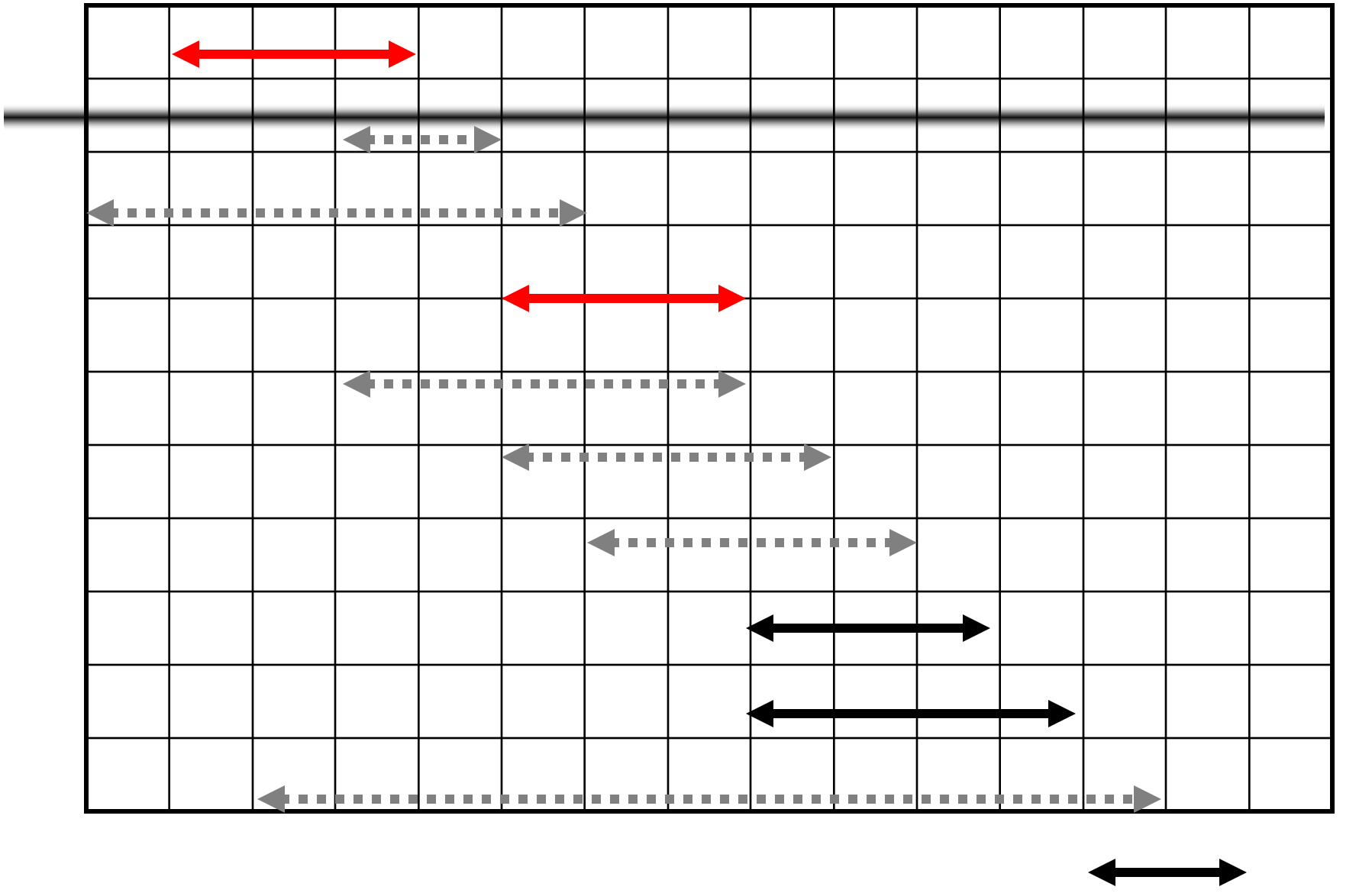
Added in optimal Solution

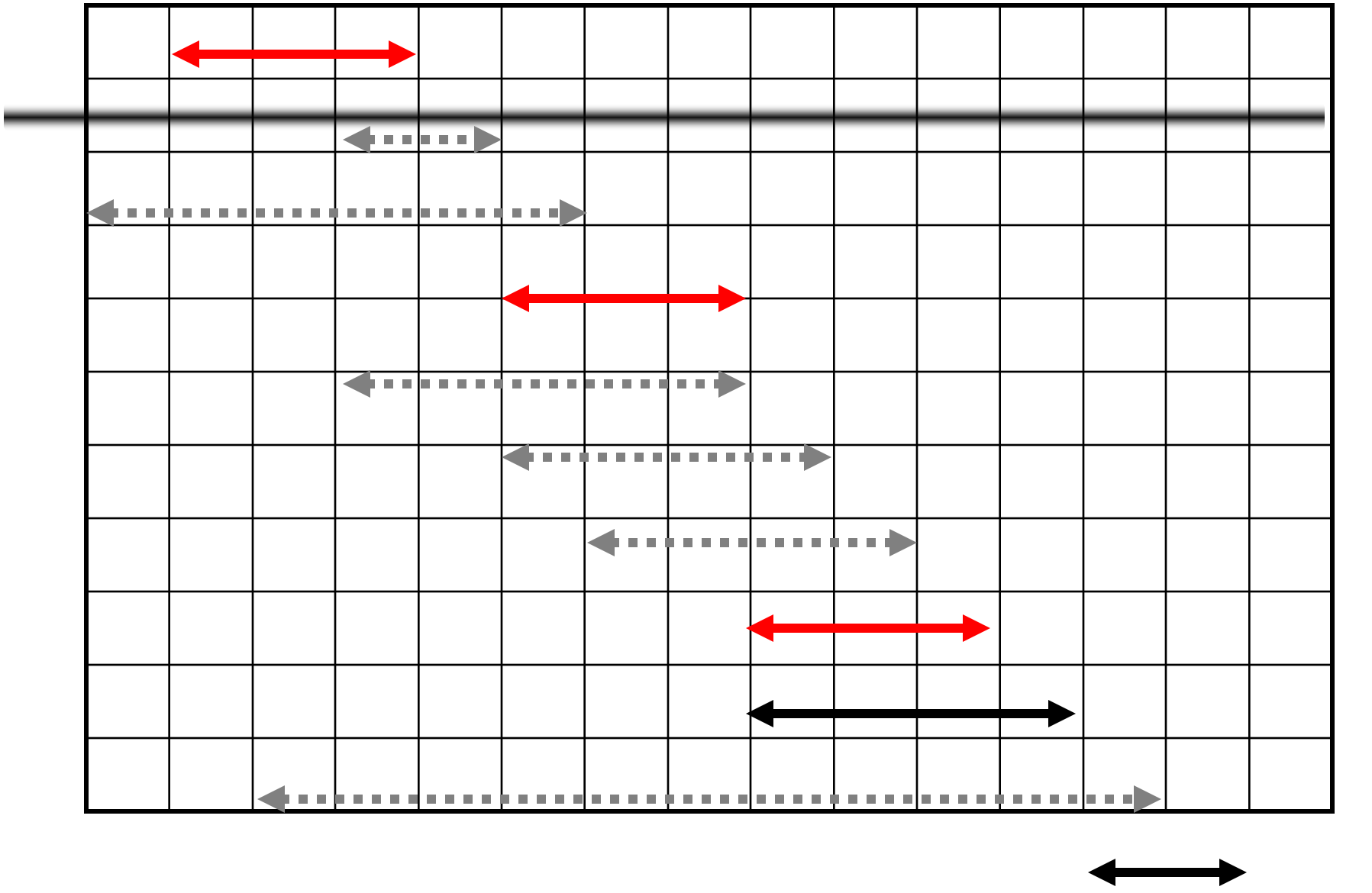
Removed from the list

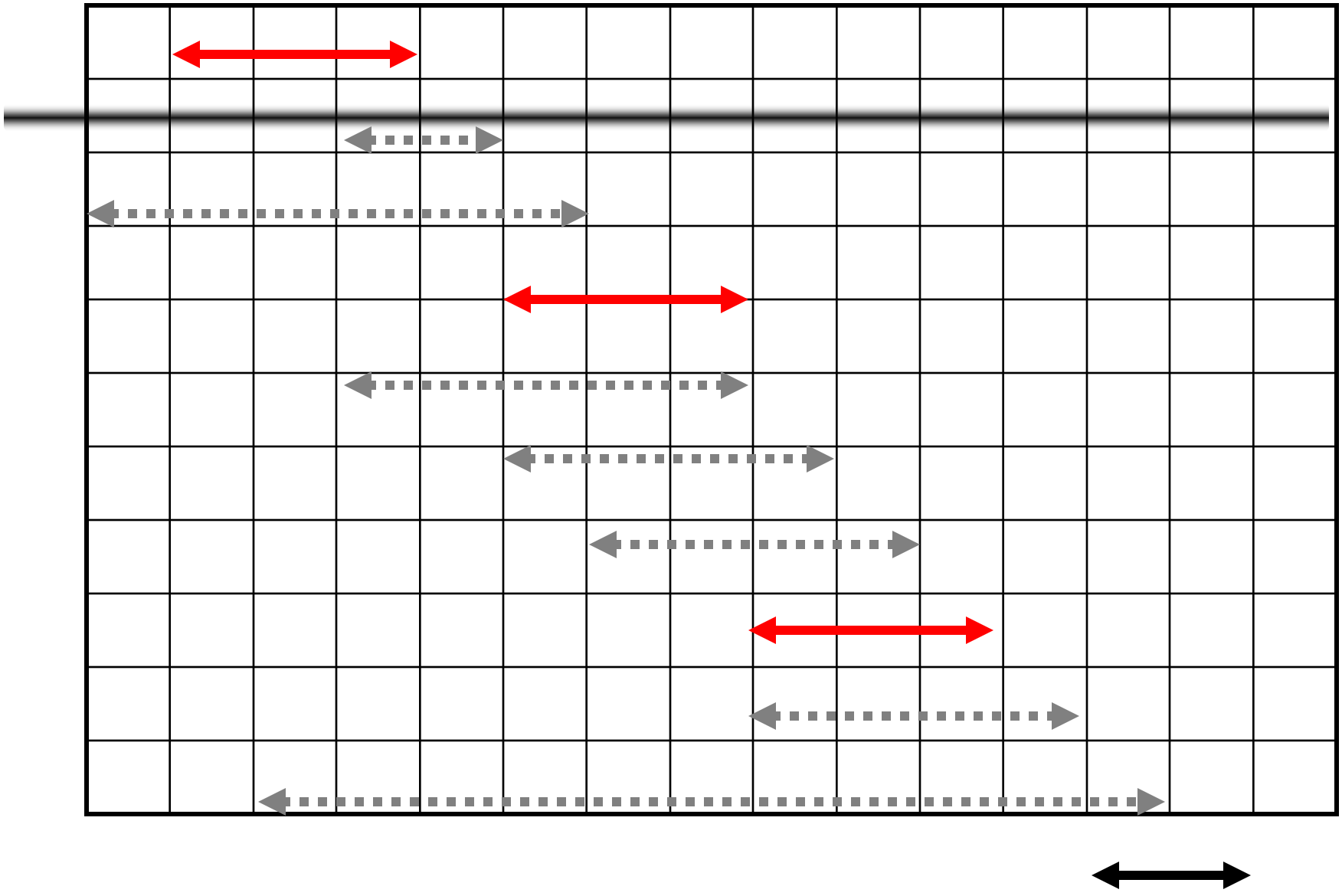

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

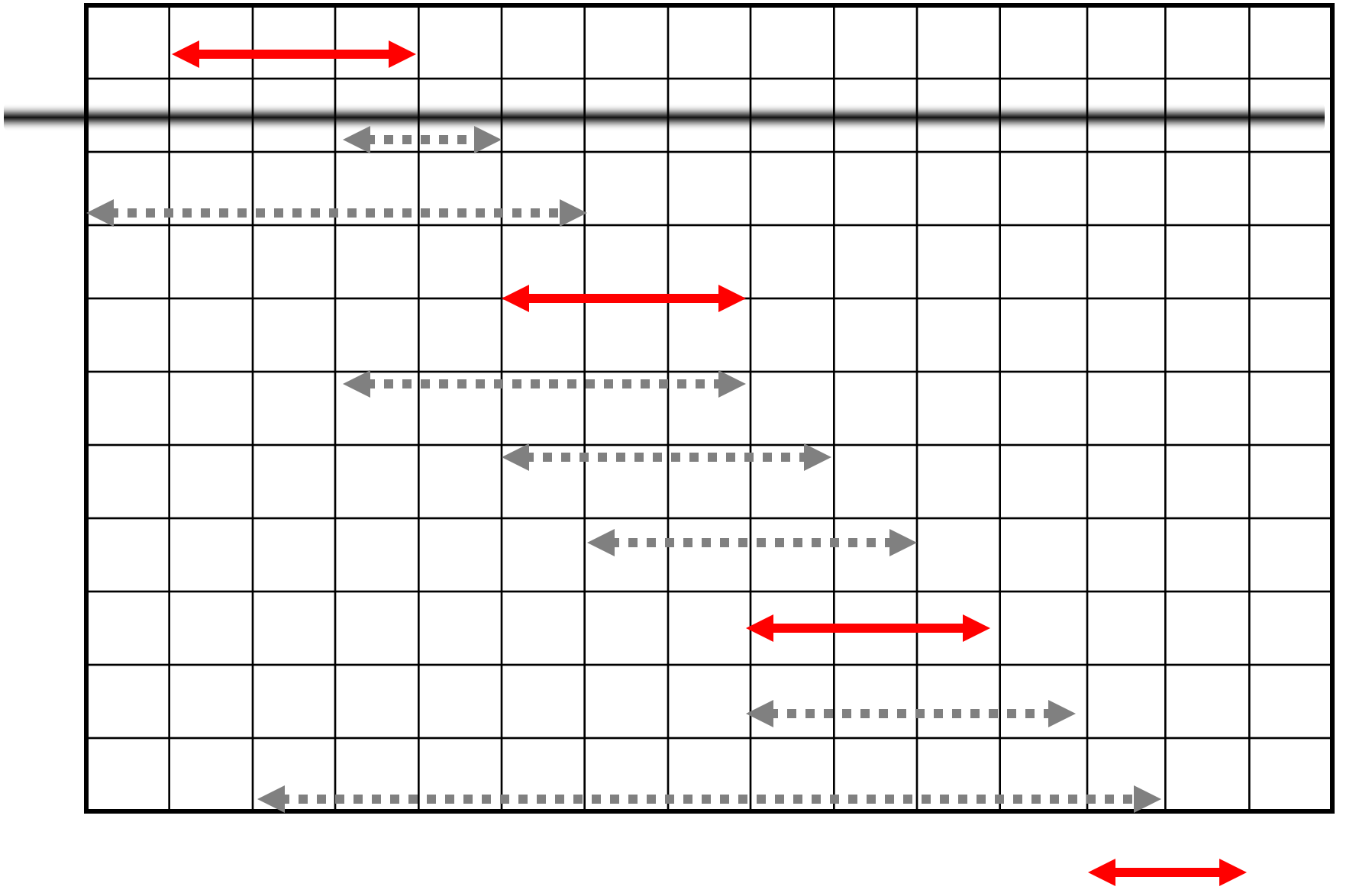

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15


0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15


0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15


0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15


0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15


0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Why this Algorithm is Optimal?

- We will show that this algorithm uses the following properties
  - The problem has the optimal substructure property
  - The algorithm satisfies the greedy-choice property
- Thus, it is Optimal

Greedy-Choice Property

- Show there is an optimal solution that begins with a greedy choice (with activity 1, which as the earliest finish time)
- Suppose $A \subseteq S$ in an optimal solution
  - Order the activities in A by finish time. The first activity in A is k
    - If k = 1, the schedule A begins with a greedy choice
    - If $k \neq 1$, show that there is an optimal solution B to S that begins with the greedy choice, activity 1
  - Let $B = A - \{k\} \cup \{1\}$
    - $f_1 \leq f_k$ → activities in B are disjoint (compatible)
    - B has the same number of activities as A
    - Thus, B is optimal

Optimal Substructures

- Once the greedy choice of activity 1 is made, the problem reduces to finding an optimal solution for the activity-selection problem over those activities in S that are compatible with activity 1
  - Optimal Substructure
  - If A is optimal to S, then $A' = A - \{1\}$ is optimal to $S' = \{i \in S: s_i \geq f_1\}$
  - Why?
    - If we could find a solution B' to S' with more activities than A', adding activity 1 to B' would yield a solution B to S with more activities than A ➔ contradicting the optimality of A
- After each greedy choice is made, we are left with an optimization problem of the same form as the original problem
  - By induction on the number of choices made, making the greedy choice at every step produces an optimal solution

Example of Greedy Algorithm

- Fractional Knapsack (Sahni)
- Huffman Coding
- Activity Selection Problem
- Minimum Spanning Tree
- Dijkstra's Shortest Path Algorithm
- Exercise: Cormen 16-1 (pg-402)